

Interacting with RESTful APIs

In this part, we will build a page that lists blog posts fetched from a REST API, as well as also a form to submit blog posts to the REST API. Through this, we will learn about various approaches to interacting with a REST API from a React component.

The approach will be using React's `useEffect` hook with the browser's `fetch` function. As part of this process, we learn how to use a type assertion function to strongly type the data from a REST API.

So, in this part, we'll cover the following topics:

- Getting set up
- Using the effect hook with fetch
- Posting data with fetch

Technical requirements

We will use the following technologies in this part:

Node.js and **npm**
Visual Studio Code

Creating the project

In this section, we will start by creating the project for the app we will build. We will then create a REST API for the app to consume.

Setting up the project

We will develop the app using Visual Studio Code and require a new Create React App-based project setup. We've previously covered this several times, so we will not cover the steps in this part – instead, see , *Setting Up React and TypeScript*.

We will style the app with Tailwind CSS. We have previously covered how to install and configure Tailwind in Create React App in . So, after you have created the React and TypeScript project, install and configure Tailwind.

We will use React Router to load data

We will use **React Hook Form** to implement the form that creates blog posts, and the `@tailwindcss/forms` plugin to style the form. See , for a reminder of how to implement these.

Understanding the component structure

The app will be a single page containing a form for adding new posts above a list of all the existing posts. The app will be structured into the following components:

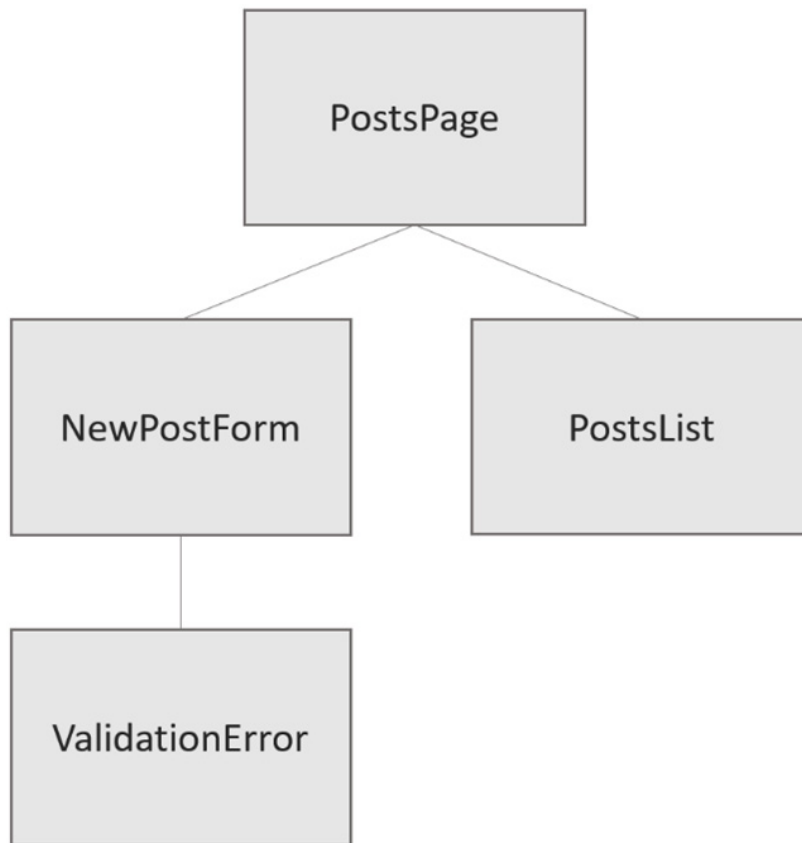


Figure 9.1 – App component structure

Here's a description of these components:

- **PostsPage** will render the whole page by referencing the **NewPostForm** and **PostsLists** components. It will also interact with the REST API.
- **NewPostForm** will render a form that allows a user to enter a new blog post. This will use the **ValidationError** component to render validation error messages. The **ValidationError** component will be the same as the one created earlier .
- **PostsList** will render the list of blog posts.

Right, now we know the component structure, let's create the REST API.

Creating a REST API

We will create a REST API using a tool called **JSON Server**, which allows a REST API to be quickly created. Install JSON Server by running the following command:

```
npm i -D json-server
```

We then define the data behind the API in a JSON file. Create a file called **db.json** in the root of the project containing the following:

```
{
  "posts": [
    {
      "title": "Getting started with fetch",
      "description": "How to interact with backend APIs using fetch",
      "id": 1
    },
    {
      "title": "Getting started with useEffect",
      "description": "How to use React's useEffect hook for interacting with b",
      "id": 2
    }
  ]
}
```

The preceding JSON means that the data behind the API will initially contain two blog posts from).

Now we need to define an npm script to start the JSON server and handle requests. Open **package.json** and add a script called **server** as follows:

```
{
  ...,
  "scripts": {
    ...,
    "server": "json-server --watch db.json --port 3001 --delay 2000",
  },
  ...
}
```

The script starts the JSON server and watches the JSON file we just created. We have specified that the API runs on port **3001** so that it doesn't clash with the app running on port **3000**. We have also slowed down the

API responses by adding a 2-second delay, which will help us see when data is being fetched from the React app.

In a terminal, start the API by running the script we just created, as follows:

```
npm run server
```

After a few seconds, the API starts. To check that the API is working correctly, open a browser and enter the following address:

http://localhost:3001/posts. The blog post data should appear in the browser as follows:

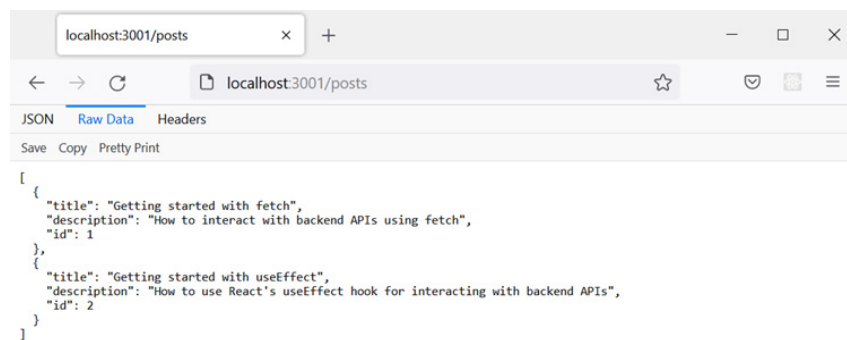


Figure 9.2 – Blog posts REST API

Now that the project is set up with a REST API, keeping the API running, next, we will learn how to interact with the REST API using **useEffect**.

Using the effect hook with fetch

In this section, we will create a page that lists the blog posts returned from the REST API we just created. We will use the browser's **fetch** function and React's **useEffect** hook to interact with the REST API.

Getting blog posts using fetch

We will start by creating a function that gets blog posts from the REST API using the browser's **fetch** function; we will store the API URL in an environment variable. To do this, carry out the following steps:

1. The same URL will be used to get, as well as save, new blog posts to the REST API. We will store this URL in an environment variable. So, create a file called **.env** in the root of the project containing this variable, as follows:

```
REACT_APP_API_URL = http://localhost:3001/posts/
```

This environment variable is injected into the code at build time and can be accessed by code using `process.env.REACT_APP_API_URL`.

Environment variables in Create React App projects must be prefixed with `React_APP_`

2. Now, create a folder called `posts` in the `src` folder for all the files for the blog post feature.
3. Create a file called `getPosts.ts` in the `posts` folder. In this file, add the following function that gets the blog posts:

```
export async function getPosts() {  
  
    const response = await fetch(  
  
        process.env.REACT_APP_API_URL!  
  
    );  
  
    const body = await response.json()  
  
    return body;  
  
}
```

The `fetch` function has an argument for the URL to the REST API. We have used the `REACT_APP_API_URL` environment variable to specify this URL. Environment variable values can be `undefined`, but we know this isn't the case, so we have added a **not null assertion** (!) after it.

NOTE

A not null assertion operator is a special operator in TypeScript. It is used to inform the TypeScript compiler that the expression before it can't be `null` or `undefined`.

`fetch` returns a `Response` object and we call its `json` method to get the response body in JSON format. The `json` method is asynchronous, so we need to `await` it.

That completes an initial version of **getPosts**. However, the return value type from **getPosts** is currently **any**, which means no type checking will occur on it. We will improve this next.

Strongly typing response data

In *Introducing TypeScript*, we learned how to make unknown data strongly typed using the **unknown** type and type predicates. We will use the **unknown** type with a slightly different TypeScript feature called a **type assertion function** to type the response data in the **getPosts** function. Carry out the following steps:

1. Add a type assertion to the JSON response so that the **body** variable has a type of **unknown**:

```
export async function getPosts() {  
  
    const response = await fetch(postsUrl);  
  
    const body = (await response.json()) as unknown;  
  
    return body;  
  
}
```

2. Next, add the following type assertion function beneath **getPosts**:

```
export function assertIsPosts(  
  
    postData: unknown  
  
    ): asserts postData is PostData[] {  
  
}
```

Notice the return type annotation: **asserts postData is PostData[]**. This is called an **assertion signature** and specifies that the **postData** parameter is of the **PostData[]** type if no error occurs in the function execution.

Don't worry about the compile error where **PostData** is referenced – we will create the **PostData** type in *step 8*.

3. Let's carry on with the implementation of **assertIsPosts**. It will be a series of checks on the **postData** parameter and it will throw an exception if a check fails. Start the implementation by checking that **postData** is an array:

```
export function assertIsPosts(  
  
    postData: unknown  
  
): asserts postData is PostData[] {  
  
    if (!Array.isArray(postData)) {  
  
        throw new Error("posts isn't an array");  
  
    }  
  
    if (postData.length === 0) {  
  
        return;  
  
    }  
  
}
```

4. Now, let's do a check on the array items to see whether they have an **id** property:

```
export function assertIsPosts(  
  
  postData: unknown  
  
): asserts postData is PostData[] {  
  
  ...  
  
  postData.forEach((post) => {  
  
    if (!('id' in post)) {  
  
      throw new Error("post doesn't contain id");  
  
    }  
  
    if (typeof post.id !== 'number') {  
  
      throw new Error('id is not a number');  
  
    }  
  
  });  
  
}
```


We loop around all the posts using the array's **forEach** method. Inside the loop, we check that the **id** property exists using the **in** operator. We also check that the **id** value is of the **number** type using the **typeof** operator.

5. We can do similar checks for the **title** and **description** properties:

```
export function assertIsPosts(

    postData: unknown

): asserts postData is PostData[] {

    ...

    postData.forEach((post) => {

        ...

        if ('title' in post) {

            throw new Error("post doesn't contain title");

        }

        if (typeof post.title !== 'string') {

            throw new Error('title is not a string');

        }

    })
}
```

```

    if '!(\'description\' in post)) {

        throw new Err`r("post do\'sn\'t contain      description");

    }

    if (typeof post.description !== \'string\') {

        throw new Err`r(\'description is not a string\');

    }

    });

}

```

That completes the implementation of the type assertion function.

6. Moving back to **getPosts**, add a call to the **assert** function:

```

export async function getPosts() {

    const response = await fetch(postsUrl);

    const body = (await response.json()) as unknown;

    assertIsPosts(body);
}

```

```
    return body;

}
```

The **body** variable will now be of the **PostData[]** type after a successful call to **assertIsPosts**. You can hover over the **body** variable in the return statement to confirm this.

7. The final steps are to add the **PostData** type. Add an import statement at the top of **getPosts.ts** as follows to import **PostData**:

```
import { PostData } from './types';
```

The file will still have compile errors because the **types** file doesn't exist yet – we'll do this in the next step.

8. Add a file called **types.ts** in the **posts** folder with the following definition for the **PostData** type:

```
export type PostData = {

    id: number;

    title: string;

    description: string;

};
```

This type represents a blog post from the REST API.

Now, we have a strongly typed function that gets blog posts from the REST API. Next, we will create a React component that lists the blog posts.

Creating a blog posts list component

We will create a React component that takes in the blog post data and renders it in a list. Carry out the following steps:

1. Create a file called **PostsList.tsx** in the **posts** folder with the following import statement:

```
import { postData } from './types';
```

2. Next, start to implement the component as follows:

```
type Props = {  
  
  posts: postData[];  
  
};  
  
export function PostsList({ posts }: Props) {  
  
  }
```

The component has a prop called **posts** that will contain the blog posts.

3. Now, render the blog posts in an unordered list, as follows:

```
export function PostsList({ posts }: Props) {  
  
  return (  
  
    <ul className="list-none">  
  
      {posts.map((post) => (  
  
        <li key={post.id} className="border-b py-4">  
  
          <h3 className="text-slate-900 font-bold">
```

```

        {post.title}

      </h3>

      <p className=" text-slate-900 ">{post.description}</p>

    </li>

  )}

</ul>

);

}

```

The Tailwind CSS classes add gray lines between the blog posts with bold titles.

That completes the **PostsList** component. Next, we will create a page component that references the **PostsList** component.

Creating a blog posts page component

We will create a blog posts page component that gets blog post data using the **getPosts** function and renders it using the **PostsList** component we just created. Carry out the following steps:

1. Create a file called **PostsPage.tsx** in the **posts** folder components with the following import statements:

```

import { useEffect, useState } from 'react';

import { getPosts } from '../getPosts';

```

```
import { postData } from './types';
```

```
import { PostsList } from './PostsList';
```

We have imported the **getPosts** function, the **PostList** component, and the **postData** type we created in the last section. We have also imported the **useState** and **useEffect** hooks from React. We will use React state to store the blog posts and use **useEffect** to call **getPosts** when the page component is mounted.

2. Start the implementation of the page component by defining the state for the blog posts and whether they are being fetched:

```
export function PostsPage() {  
  
  const [isLoading, setIsLoading] = useState(true);  
  
  const [posts, setPosts] = useState<postData[]>([]);  
  
}
```

3. Next, call the **getPosts** function using the **useEffect** hook as follows:

```
export function PostsPage() {  
  
  ...  
  
  useEffect(() => {  
  
    let cancel = false;  
  
    getPosts().then((data) => {
```

```

        if (!cancel) {

            setPosts(data);

            setIsLoading(false);

        }

    });

    return () => {

        cancel = true;

    };

}, []);

}

```

We use the older promise syntax when calling **getPosts** because the newer **async/await** syntax can't be directly used within **useEffect**.

If the **PostsPage** component is unmounted while the call to **getPosts** is still in progress, the setting of the **data** and **isLoading** state variables will result in an error. For this reason, we have used a **cancel** flag to ensure that the component is still mounted when the **data** and **isLoading** state variables are set.

We have also specified an empty array as the effect dependencies so that the effect only runs when the component is mounted.

4. Add a loading indicator while the data is being fetched after the call to `useEffect`:

```
export function PostsPage() {  
  
  ...  
  
  useEffect(...);  
  
  if (isLoading) {  
  
    return (  
  
      <div className="w-96 mx-auto mt-6">  
  
        Loading ...  
  
      </div>  
  
    );  
  
  }  
  
}
```

The Tailwind CSS classes position the loading indicator horizontally in the center of the page with a bit of margin above it.

5. Finally, render a page title and the posts list after the conditional loading indicator:


```
export function PostsPage() {

  ...

  if (isLoading) {

    return (

      <div className="w-96 mx-auto mt-6">

        Loading ...

      </div>

    );

  }

  return (

    <div className="w-96 mx-auto mt-6">

      <h2 className="text-xl text-slate-900 font-bold">Posts</h2>

      <PostsList posts={posts} />

    </div>

  );
}
```

```
</div>
```

```
);
```

```
}
```

The Tailwind CSS classes position the list in the center of the page with a bit of margin above. A large **Posts** title is also rendered above the list in a dark gray color.

6. Now, open **App.tsx** and replace its contents with the following so that it renders the page component we just created:

```
import { PostsPage } from './posts/PostsPage';
```

```
function App() {
```

```
  return <PostsPage />;
```

```
}
```

```
export default App;
```

7. Run the app by running **npm start** in a new terminal separate from the one running the REST API. The loading indicator will appear briefly as the data is being fetched:

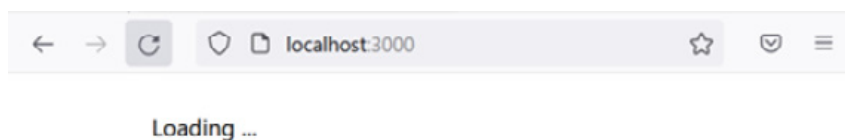


Figure 9.3 – Loading indicator

The blog post list will then appear as follows:

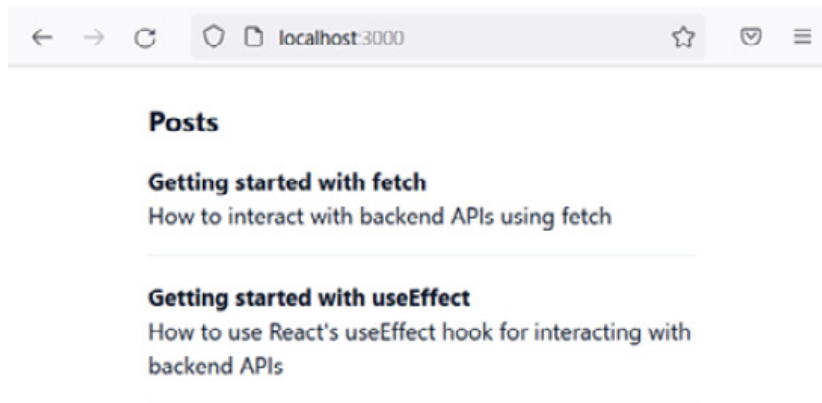


Figure 9.4 – Blog posts list

That completes this version of the **PostsPage** component.

Here are the key points we learned in this section on interacting with HTTP **GET** requests in a REST API using **fetch** and **useEffect**:

- **fetch** will make the actual HTTP request that has the REST API's URL as a parameter
- A type assertion function can be used to strongly type response data
- **useEffect** can trigger the **fetch** call when the component that holds the data in state is mounted
- A flag can be used inside **useEffect** to check that the component hasn't been unmounted during the HTTP request before the data state is set

Still keeping the app and REST API running, in the next section, we will learn how to post data to a REST API using **fetch**.

Posting data with fetch

In this section, we will create a form that submits a new blog post to our REST API. We will create a function that uses **fetch** to post to the REST API. That function will be called in the form's submit handler.

Creating new blog posts using fetch

We will start by creating the function that sends a new blog post to the REST API. This will use the browser's **fetch** function, but this time, using an HTTP **POST** request. Carry out the following steps:

1. We will start by opening **types.ts** in the **posts** folder and adding the following two types:

```
export type NewPostData = {
```

```
  title: string;
```

```

    description: string;

};

export type SavedPostData = {

    id: number;

};

```

The first type represents a new blog post, and the second type represents the data from the API when the blog post is successfully saved.

2. Create a new file called **savePost.ts** in the **posts** folder and add the following import statement:

```
import { NewPostData, SavedPostData } from './types';
```

We have also imported the types we just created.

3. Start to implement the **savePost** function as follows:

```

export async function savePost(

    newPostData: NewPostData

) {

    const response = await fetch(

        process.env.REACT_APP_API_URL!,

```

```

{

    method: 'POST',

    body: JSON.stringify(newPostData),

    headers: {

        'Content-Type': 'application/json',

    },

}

);

}

```

The **savePost** function has a parameter, **newPostData**, containing the title and description for the new blog post, and sends it to the REST API using **fetch**. A second argument has been specified in the **fetch** call to specify that an HTTP **POST** request should be used and that the new blog post data should be included in the request body. The request body has also been declared as being in JSON format.

4. Next, strongly type the response body as follows:

```

export async function savePost(newPostData: NewPostData) {

    const response = await fetch( ... );

```

```

    const body = (await response.json()) as unknown;

    assertIsSavedPost(body);

}

```

We set the response body as having the **unknown** type and then use a type assertion function to give it a specific type. This will raise a compile error until we implement **assertIsSavedPost** in *step 6*.

5. Finish the implementation of **savePost** by merging the blog post ID from the response with the blog post title and description supplied to the function:

```

export async function savePost(newPostData: NewPostData) {

    ...

    return { ...newPostData, ...body };

}

```

So, the object returned from the function will be a new blog post with the ID from the REST API.

6. The last step is to implement the type assertion function:

```

function assertIsSavedPost(

    post: any

): asserts post is SavedPostData {

```

```

    if (!('id' in post)) {

        throw new Error("post doesn't contain id");

    }

    if (typeof post.id !== 'number') {

        throw new Error('id is not a number');

    }

}

```

The function checks whether the response data contains a numeric **id** property, and if it does, it asserts that the data is of the **SavedPostData** type.

That completes the implementation of the **savePost** function. Next, we will add a form component that allows the user to enter new blog posts.

Creating a blog post form component

We will create a component that contains a form that captures a new blog post. When the form is submitted, it will call the **savePost** function we just created.

We will use React Hook Form to implement the form along with a **ValidationError** component. We covered React Hook Form and the **ValidationError** component in detail earlier , so the implementation steps won't be covered in much detail.

Carry out the following steps:

1. We will start by creating a **ValidationError** component that will render form validation errors. Create a file called **ValidationError.tsx** in the **posts** folder with the following content:

```
import { FieldError } from 'react-hook-form';

type Props = {

  fieldError: FieldError | undefined,

};

export function ValidationError({ fieldError }: Props) {

  if (!fieldError) {

    return null;

  }

  return (

    <div role="alert" className="text-red-500 text-xs mt-1">

      {fieldError.message}

    </div>

  );
```



```
}
```

2. Create a new file in the **posts** folder called **NewPostForm.tsx**. This will contain a form to capture the title and description for a new blog post. Add the following import statements to the file:

```
import { FieldError, useForm } from 'react-hook-form';
```

```
import { ValidationError } from './ValidationError';
```

```
import { NewPostData } from './types';
```

3. Start to implement the form component as follows:

```
type Props = {
```

```
  onSave: (newPost: NewPostData) => void;
```

```
};
```

```
export function NewPostForm({ onSave }: Props) {
```

```
}
```

The component has a prop for saving a new blog post so that the interaction with the REST API can be handled outside of this form component.

4. Now, destructure the **register** and **handleSubmit** functions and useful state variables from React Hook Form's **useForm** hook:

```
type Props = {
```

```
  onSave: (newPost: NewPostData) => void;
```

```
};
```

```
export function NewPostForm({ onSave }: Props) {
```

```
  const {
```

```
    register,
```

```
    handleSubmit,
```

```
    formState: { errors, isSubmitting, isSubmitSuccessful },
```

```
  } = useForm<NewPostData>();
```

```
}
```

We pass the type for the new post data into the **useForm** hook so it knows the shape of the data to capture.

5. Create a variable for the field container style and a function for the editor style:

```
export function NewPostForm({ onSave }: Props) {
```

```
  ...
```

```
  const fieldStyle = 'flex flex-col mb-2';
```

```
  function getEditorStyle()
```

```

      fieldError: FieldError | undefined

    ) {

      return fieldError ? 'border-red-500' : '';

    }

  }

```

6. Render **title** and **description** fields in a **form** element as follows:

```

export function NewPostForm({ onSave }: Props) {

  ...

  return (

    <form

      noValidate

      className="border-b py-4"

      onSubmit={handleSubmit(onSave)}

    >

```

```
<div className={fieldStyle}>
```

```
<label htmlFor="title">Title</label>
```

```
<input
```

```
  type="text"
```

```
  id="title"
```

```
  {...register('title', {
```

```
    required: 'You must enter a title',
```

```
  })}
```

```
  className={getEditorStyle(errors.title)}
```

```
<ValidationError fieldError={errors.title} />
```

```
</div>
```

```
<div className={fieldStyle}>
```

```
<label htmlFor="description">Description</label>
```

```
<textarea
```

```
id="description"
```

```
{...register('description', {
```

```
  required: 'You must enter the description',
```

```
}}}
```

```
className={getEditorStyle(errors.description)}
```

```
/>
```

```
<ValidationError fieldError={errors.description} />
```

```
</div>
```

```
</form>
```

```
);
```

```
}
```

7. Lastly, render a **Save** button and the success message:

```
<form
```

```
noValidate
```

```
className="border-b py-4"
```

```
onSubmit={handleSubmit(onSave)}
```

```
>
```

```
<div className={fieldStyle}> ... </div>
```

```
<div className={fieldStyle}> ... </div>
```

```
<div className={fieldStyle}>
```

```
<button
```

```
type="submit"
```

```
disabled={isSubmitting}
```

```
className="mt-2 h-10 px-6 font-semibold bg-black text-white"
```

```
>
```

Save

</button>

{isSubmitSuccessful && (

<div

role="alert"

className="text-green-500 text-xs mt-1"

>

The post was successfully saved

</div>

}}

</div>

</form>

That completes the implementation of the **NewPostForm** component.

8. Now open **PostPage.tsx** and import the **NewPostForm** component and the **savePost** function we created earlier. Also, import the **NewPostData** type:

```
import { useEffect, useState } from 'react';
```

```
import { getPosts } from './getPosts';
```

```
import { postData, NewPostData } from './types';
```

```
import { PostsList } from './PostsList';
```

```
import { savePost } from './savePost';
```

```
import { NewPostForm } from './NewPostForm';
```

9. In the **PostPage** JSX, add the **NewPostForm** form above the **PostsList**:

```
<div className="w-96 mx-auto mt-6">
```

```
  <h2 className="text-xl text-slate-900 font-bold">Posts</h2>
```

```
  <NewPostForm onSave={handleSave} />
```

```
  <PostsList posts={posts} />
```

```
</div>;
```

10. Add the save handler function just below the effect that gets blog posts:

```
useEffect(() => {
```

```
  ...
```



```
}, []));
```

```
async function handleSave(newPostData: NewPostData) {
```

```
    const newPost = await savePost(newPostData);
```

```
    setPosts([newPost, ...posts]);
```

```
}
```

11. The handler calls **savePost** with the data from the form. After the post has been saved, it is added to the start of the **posts** array.
12. In the running app, the new blog post form will appear above the blog post list, as follows:

Posts

Title

Description

Save

Getting started with fetch

How to interact with backend APIs using fetch

Getting started with useEffect

How to use React's useEffect hook for interacting with backend APIs

Figure 9.5 – New blog post form above the posts list

13. Fill in the form with a new blog post and press the **Save** button. The new post should appear at the top of the list after a couple of seconds.

Posts

Title

Using fetch to POST data

Description

How to use fetch to make HTTP POST requests to a backend API

Save

The post was successfully saved

Using fetch to POST data
How to use fetch to make HTTP POST requests to a backend API

Getting started with fetch
How to interact with backend APIs using fetch

Getting started with useEffect
How to use React's useEffect hook for interacting with backend APIs

Figure 9.6 – New blog post added to posts list

That completes the implementation of the form and its integration into the blog posts page.

Here are a couple of key points we learned in this section on posting data with **fetch**:

- The second parameter in the **fetch** function allows the HTTP method to be specified. In this section, we used this parameter to make an HTTP **POST** request.
- The second parameter in **fetch** also allows the request body to be supplied.

Once again, keeping the app and REST API running, Our app is complete,

,